

Invited Paper: Physics-Driven Real-Time CPS Challenge

Paolo Pazzaglia, Kevin Schmidt, Laura Beermann, Dirk Ziegenbein and Arne Hamann
Robert Bosch GmbH, Corporate Research, Germany. Emails: *name.surname@de.bosch.com*

Abstract—Traditional real-time approaches often ignore the dynamic, state-dependent nature of (functional) requirements in distributed Cyber-Physical Systems (CPSs), leading to overly pessimistic design solutions, which either significantly over-provision computational and communication resources, or result in inferior functional performance. This Challenge aims to foster fundamental discussions and collaborations at the intersection of real-time systems and control engineering. We propose to explore a paradigm shift: moving beyond static, worst-case real-time attributes towards dynamic, context-aware ones, rooted in physics-driven performance. We provide a realistic automotive use-case, featuring a distributed lateral motion control functionality, and challenge the real-time community to develop innovative scheduling and design solutions. The ultimate goal is to facilitate more efficient, robust, and adaptive distributed applications, by aligning real-time system design with the dynamic needs of control, without over-dimensioning resources.

Index Terms—Distributed Control, Real-Time, Cyber-Physical Systems, Industrial Challenge

I. INTRODUCTION

In the real-time community, functionality is typically completely abstracted away by the established system modeling practices, while the focus is mostly targeted to meeting timing requirements regarding various deadline models. The most commonly used models assume so-called hard deadlines, where no violations are permitted, or weakly-hard deadlines [1], in which a certain number of violations are allowed within specific time windows. This becomes very evident when we examine the questions of a well-known survey issued to industrial practitioners [2]. Accordingly, a review of the current state of the art in real-time systems suggests an increasing gap between academic research and systems deployed in industry. As a result, research outcomes face challenges when being transferred to industrial settings.

The limitations of traditional models are especially clear in domains and systems that involve real-time feedback control [3]: While classic timing models define a fixed characterization, in these systems behavior and dynamics change with environmental conditions. The primary goal is thus designing control algorithms that ensure safe and correct operation, while also providing meaningful performance guarantees despite uncertainties that can impact the physical dynamics, such as environmental changes, variations in physical parameters (e.g., through wear and tear), and timing fluctuations [4]. Requiring strict adherence to requirements of chosen (fixed) deadline models is, thus, just addressing parts of the challenge and might overly restrict the overall solution space towards inefficient and expensive systems.

It should be noted that the impact of each source of uncertainty on the control performance is highly dependent on the system and its operational context. Consequently, timing variations may be considered negligible in certain scenarios, for example, in the case when the sensitivity to physical uncertainties exceeds the one to temporal variations by orders of magnitude, or when the plant exhibits low-pass characteristics, rendering it insensitive to high-frequency timing jitter. Conversely, in other cases, strict and precise timing is of greater importance, for instance, when rapid responses to events are required or if the system exhibits inherent instability.

The above discussion highlights a critical weakness in many current methodologies: While analytically attractive, they often fail to address the complex interplay between the various facets of uncertainty and system dynamics. Consequently, building truly effective and efficient systems requires a paradigm shift. Rather than treating timing and functional aspects as isolated problems, a co-design [5] approach that considers them concurrently is essential for achieving optimal and reliable performance. Parts of the research community have already started exploring these concepts. Early examples proposed, e.g., dynamic scheduling [6], [7], [8], resource allocation [9], [10] and period adaptation [11] based on feedback signals from the controlled plant, and many more recent works following. Multiple co-simulation tools have been proposed, such as Jitterbug and Truetime [12], TRES [13], Jittertime [14] and frameworks based on Simulink [15] and Functional Mockup Interface (FMI) [16]. Additionally, control performance and safety criteria are increasingly used to characterize the effects of timing on safe system operation, exceeding pure binary pass/fail systematics, see e.g., [17], [14], [18], [19].

This becomes even more obvious with the evolution of CPSs towards edge and cloud-distributed architectures, which presents both new challenges and opportunities for control engineering [20], [21]. The vision, where functionalities are executed on such distributed architectures, promises significant benefits in terms of cost, energy efficiency, and flexibility [22] and is a key enabler for advanced applications like flexible manufacturing, Software-Defined Vehicles (SDVs), and infrastructure-based automated driving. The increased connectivity and compute capabilities enable novel advanced control methods and applications utilizing multiple sources of information to meet challenging control objectives [23].

However, this paradigm shift also introduces or exacerbates so-called “timing non-idealities” such as network-induced delays, jitter, packet losses, and additional temporal variability,

whose proper characterization in controlled systems is still an open research topic, especially for time-critical systems [24]. In particular, when temporal variations are aligned to critical transient conditions in the controlled physical system, stability and performance metrics may be severely degraded [25]. These effects act in addition to other sources of uncertainties already present in μ C-based compute nodes, and may compound in complex ways through functional chains of tasks [26], [27].

We highlight a critical need in modern control system development: *moving beyond static, worst-case real-time assumptions towards solutions that are aware of the dynamic interplay between control objectives, distributed system characteristics, and varying timing and physical requirements*. To this end, we introduce a new Challenge, aimed at integrating CPS aspects in the design and runtime management of distributed real-time systems. We provide a simulator of a lateral motion controller of a vehicle, partially deployed in the cloud, where the functionality is split among a chain of real-time tasks with input-output dependencies. Such system can undergo both timing uncertainties and variable driving conditions with different criticality. The users of this Challenge are asked to find optimal policies to efficiently accommodate multiple instances of such functionalities in the cloud, using the online control performance monitors to drive the adaptation at runtime. The Challenge sets the following targets:

- **Linking real-time and control performance:** We consider applications where the computational and communication delays may be significant enough to affect the functional CPS behavior. At the same time, information about the physical system status can and should be leveraged to adapt the real-time constraints at runtime.
- **Going beyond hard real-time:** Many relevant industrial control systems are not inherently “hard real-time” across all their functions, and can tolerate variations in timing and data freshness without compromising stability or performance guarantees.
- **Defining dynamic and context-aware requirements:** The real-time requirements of many CPSs are not necessarily static; the timing constraints within which a computation must be completed, or a value must be transmitted, change dynamically based on the *context*, which includes the state of the plant, the environment (e.g., network traffic), and even the chosen control approach (e.g., adaptive, robust).
- **Optimizing resource utilization:** Over-provisioning resources based on worst-case scenarios leads to costly solutions. By understanding and exploiting the dynamic nature of timing requirements, we can design more flexible, resource-efficient and sustainable systems.
- **Addressing complex functional chains:** The control functionality is implemented over a chain of tasks encompassing both embedded and remote platforms, as well as network communications. How timing uncertainties experienced by one or more tasks affect the whole functional chain is still a key underexplored topic.

- **Bridging disciplinary gaps:** We aim to foster a stronger dialogue between the real-time systems and control communities, enabling control engineers to articulate their timing needs more effectively and real-time researchers to develop more effective solutions for CPS applications.

This paper is structured as follows. Section II describes how modern control methods deal with uncertainties. Section III describes the Challenge and Section IV the target questions. Section V provides a motivational example obtained using the Challenge model and Section VI summarizes the conclusions and the message to the Challenge users. Finally, Appendix A explores the technical model of the target CPS and Appendix B provides a summary and reflections about the previous Bosch Challenges.

II. MODERN REAL-TIME CONTROL SYSTEMS

This section provides a background on distributed control systems, and introduces modern methods and performance criteria designed to maintain and measure reliability and performance in the face of uncertainties.

A. Boundaries of classical control

Classical controllers are usually designed under the assumption of ideal timing (e.g., full periodicity, no jitter, constant delays etc.), with control objectives defined through specifications focused on stability, response times, overshoot, steady-state variability, and steady-state error [28]. Hard deadlines and perfect periodicity requirements have been created as conservative abstractions of timing constraints, which in turn limit also the design space for controllers, forcing larger periods for control tasks and thus generally worse performance [12], [29].

The limitation of classical approaches becomes apparent when timing variations occur [30]. Classic nominal-condition controllers may show intrinsic robustness to limited timing variations [17], but in general require extensive testing and checks when subject to different delays than modeled: Indeed, systems that operate smoothly under nominal conditions can quickly transition through progressively worse performance stages when the impact of uncertain delays increases [31].

B. Uncertainty-aware control methods

The considerations above may have created the preconception that hard deadline models and ideal timing assumptions are justified due to the requirements of *all* (or a vast majority of) control applications. Instead, the evolution of modern control theory is pushing in a direction where hard deadlines are required only for a very limited number of controllers (e.g., ABS and ESP controllers), which most likely have already a targeted embedded solution.

Modern controllers are typically formulated as *optimal control problems*, often including uncertainties about the environment and plant. Methods to deal with such uncertainties include Kalman Filtering for state estimation and parameter uncertainties, Model Predictive Control focusing on constraints, stability, performance optimization [32], and recently Reinforcement Learning approaches. Such methods

offer a fundamentally different paradigm that can handle non-idealities, also in timing, in a stochastic sense. Rather than demanding perfect system knowledge and treating any deviation as a failure condition, they can include models of uncertainties (including timing variations) and account for them in the control design [33]. Functional requirements are often defined within an acceptance range depending on system conditions (e.g., if in transient behavior or steady state operation). This shift from deterministic to probabilistic reasoning is essential for CPSs application: The goal is to seek controllers that perform optimally on average, while providing probabilistic guarantees on constraint satisfaction, explicitly acknowledging that full knowledge and ideal execution are unattainable in distributed systems.

Multiple works deal in particular with the design of controllers in edge and cloud, targeting robustness to their uncertain timing conditions [21]. Nonetheless, designs for distributed CPSs still require bounds and models on timing non-idealities, and their type and distribution have major implications. For example, dealing with oscillating or highly uncertain delays results in worse overall performance for the closed-loop CPSs than with large but more regular delays.

In such scenarios of high timing uncertainties, fitting classic hard deadline models results in highly pessimistic characterizations and conservative controller tuning. Thus, new methods from real-time engineering, on one hand to develop better timing abstractions to be integrated in control design, and on the other hand to bound timing delays and uncertainties in critical conditions, are also required to help develop better controllers. The special role of timing is further emphasized considering the *information asymmetry* inherent in distributed control: That is, while disturbances and measurement noise affect the control system but remain unknown to the controller, certain timing information can potentially be monitored or estimated [34]. This creates opportunities for adaptive strategies that explicitly exploit timing information when available, a possibility that does not exist for most other uncertainty sources.

C. Dealing with timing non-idealities in CPSs

To address timing uncertainties in CPSs, the control and real-time systems communities have developed various approaches that differ in when and how timing effects are considered. We focus on three different strategies:

1) *Facing uncertain timing implicitly*: Control systems can be made robust to variable and uncertain (but limited) delays. These approaches can deal with timing as part of the overall uncertainty, without explicitly modeling it as a separate source of non-ideality. For example, learning-based model predictive control integrates machine learning to handle uncertainties while maintaining safety [35], while Gaussian process-based methods use probabilistic models for robust performance [36].

2) *Considering timing-effects at design phase*: Controllers can be made inherently robust against variable timing effects, by incorporating models of such timing non-idealities directly into the control design process. Example of such methods encompass robustifying the control design to jitter [37], [38]

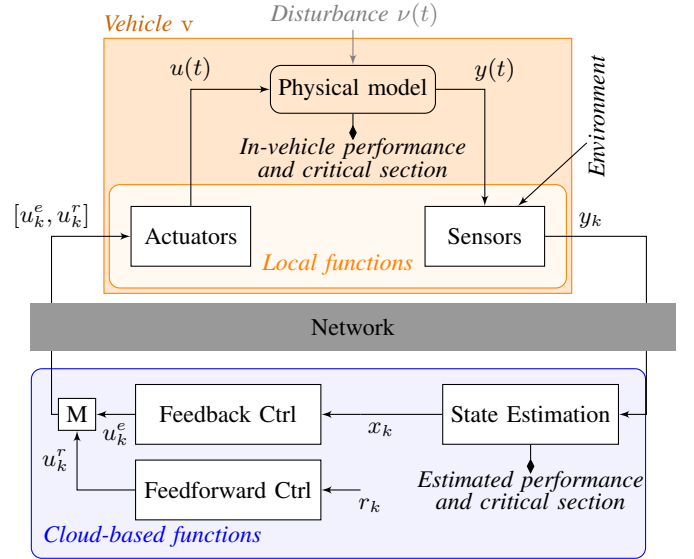


Fig. 1. The functional control chain under analysis for a vehicle v . The in-car sensing values are transmitted to the controller tasks in the cloud via network communication. The corresponding control commands (both open and closed loop) are computed, collected and then sent back to the car actuators to perform the steering. Control-related performance metrics can be evaluated both in the vehicle (with internal states) and in cloud (with estimated states).

and deadline misses [29], as well as structural approaches that optimize controller architectures for timing robustness [39], distributionally robust control techniques for systems with uncertain timing characteristics [40]. Such class of methods can provide strong performance guarantees even under significant timing uncertainties. However, it requires accurate modeling of timing effects and may lead to conservative designs if the timing model is not well-aligned with reality.

3) *Adapting to timing variations during runtime*: This approach dynamically adjusts control strategies and system behavior in response to observed timing variations during operation. Unlike design-time approaches, these methods monitor timing patterns and adapt controller parameters, scheduling policies, or system configurations in real-time. Recent examples include adaptive control approaches that modify control gains based on measured delays [20], adaptive controller parametrization for systems subject to overruns [41], [42], resource-adaptive control periods [43], [44], and MPC methods for deadline overruns [45]. This approach offers the advantage of responding to actual system conditions rather than conservative worst-case assumptions, potentially achieving better performance under varying operational conditions.

III. THE NEW CHALLENGE AND USE-CASE

This Section describes the main components provided as part of the Challenge, namely the controlled CPS, the platform architecture and the task model. Table I, at the end of the Section, serves as a recap of all fixed and tunable parameters.

A. Overview and functional model

The Challenge considers a race circuit with N_v vehicles, denoted by the set $\mathcal{V}$. Each target vehicle $v \in \mathcal{V}$ is controlled

in its lateral maneuver (steering) via a chain of functions distributed across the vehicle and the cloud, as presented in Figure 1, alongside its other functionalities.

The control chain starts with a set of sensors that sample the variables of interests $y(t)$ from the dynamics of v and the environment at periodic time instants, obtaining discretized sensing values y_k . Such values are sent to a state estimation function, which produces the corresponding vehicle states x_k . The corresponding control command u_k is then computed as a combination of feedback (u_k^e , based on x_k) and feedforward (u_k^r , based on reference trajectory r_k) components, and is sent to the actuators that perform the physical steering ($u(t)$) of the vehicle. The vehicle may be affected by disturbances and noise, modeled here as an external input $\nu(t)$. More details of the physical model are presented in Appendix A.

All components of the control chain are implemented as real-time tasks. A functional model of such control chain is provided as part of the Challenge. Sensing and actuator tasks are executed on the corresponding in-vehicle μ Cs or μ Ps, while the estimation and the lateral control tasks of all vehicle chains are executed on a shared cloud-based platform, with computational resources limited by design for cost reasons. Data between the vehicles and the cloud is exchanged via wireless network communication, possibly subject to additional delays and packet losses. We provide the functional simulation, as well as a set of parameters as fixed design starting points, while the scheduling policy is left as open design space¹.

B. Performance model and evaluation

As the vehicles traverse the circuit, they are subject to conditions of varying *criticality* and *sensitivity* to control performance, depending on the *context*, i.e., the combination of vehicle states and environment conditions. A vehicle is identified as being in a *critical section* by its spatial position on the track, while its performance is dynamically quantified, e.g., by the lateral deviation from its intended trajectory. As a rule of thumb, it is established that when driving on straight sections, the control problem is less complex and exhibits lower sensitivity to latency and temporal non-idealities. In contrast, when navigating curves, the state of the system and the control commands change at a higher frequency, resulting in greater sensitivity to delayed or skipped updates. A set of performance monitors for different control performance metrics and critical sections are implemented both in-vehicle (based on its internal states), and in the cloud (based on the transmitted/estimated vehicle state). Such metrics are in place to check given control-based constraints that must be satisfied. Exemplary performance metrics are provided in Section V-B.

A simulation of the system is carried out by instantiating a fleet of N_v vehicles (each vehicle associated to its functional model instance), each with a unique initial state defined by

¹The code and a detailed description of the overall system can be found at <https://github.com/boschresearch/CPSSChallenge>. This repository will allow to manage future extensions and possible corrections to the proposed setup. In case of discrepancies between the model presented here and the one in the Github folder description, the latter shall prevail.

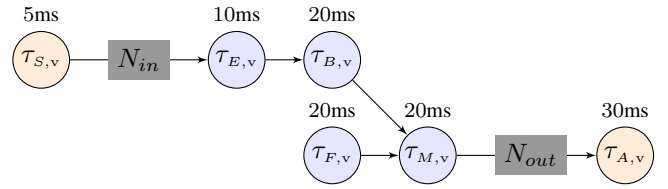


Fig. 2. DAG of the controller function for a vehicle v with example periods – supposing all tasks are periodic. Light orange tasks are in $\Gamma^L(v)$ and light blue tasks are in $\Gamma^C(v)$. Gray boxes represent communication via network.

its starting position ($x_{0,v}$) on the track and initial velocity ($v_{0,v}$). Because each vehicle begins with a unique position and velocity, the vehicles across a fleet encounter critical sections at different time points, and with different time intervals between two critical sections.

Analyzing and simulating such a system effectively requires working with a *functional* and a *timing* layer.

- The functional layer encompasses the physical behavior of the controlled system, the controller logic as well as the timed flow of data in the control chain.
- The timing layer encompasses the scheduling of the software components across the available computational and communication resources.

These two layers, while often considered separately by many state-of-the-art works, are here instead *intertwined*, representing a more realistic model of CPSs: The timing layer produces timed triggers of the data reads and writes, which are fed to the functional layer for the control performance analysis. Additionally, the timing layer should use runtime performance metrics of the physical simulation, to make informed online adaptations of scheduling and resource allocations.

The timing layer represents the core optimization target of the Challenge. For example, the scheduling strategy could be designed to boost the priorities of the tasks related to a vehicle that is experiencing degraded performance, or to adapt them when the vehicle enters and exits critical sections of the track. To this end, we provide the main timing and architectural parameters of the task set, as well as the target control performance metrics and constraints, while leaving ample freedom regarding mapping, scheduling and other design choices. A proper solution of this Challenge requires the satisfaction of all performance constraints for all N_v vehicles alongside a full track round.

C. Task chain and real-time execution model

We here formally define the model for the chain of tasks $\Gamma(v) \in \Gamma$, assigned to a vehicle $v \in \mathcal{V}$. The chain can be further divided considering the tasks executing locally in the vehicle $\Gamma^L(v)$ and in the cloud $\Gamma^C(v)$, respectively, with $\Gamma^L(v) \cup \Gamma^C(v) = \Gamma(v)$, $\Gamma^L(v) \cap \Gamma^C(v) = \emptyset$. In this model the control logic resides in $\Gamma^C(v)$ and executes within the cloud infrastructure. Sensing and actuation are performed at the vehicle edge in $\Gamma^L(v)$. Each vehicle transmit their sensed values only to its related control tasks.

A generic task $\tau_{a,v} \in \Gamma(v)$ is denoted in its general form as $\tau_{a,v} = \{T_{a,v}, D_{a,v}, C_{a,v}, \phi_{a,v}, \pi_{a,v}\}$, where $T_{a,v}$ is the

period or activation pattern (possibly different w.r.t. the other tasks in the chain), $D_{a,v}$ is the deadline, $C_{a,v}$ a model of its execution time, $\phi_{a,v}$ its initial release offset and $\pi_{a,v}$ its (possibly dynamic) priority. The control logic in Figure 1 for an arbitrary vehicle $v \in \mathcal{V}$ can be mapped to a Directed Acyclic Graph (DAG), as shown in Figure 2, comprising the periodic tasks for sensing ($\tau_{S,v}$), state estimation ($\tau_{E,v}$), feed-forward and feedback control ($\tau_{F,v}$ and $\tau_{B,v}$), aggregation and communication back to the vehicle ($\tau_{M,v}$), and actuation ($\tau_{A,v}$) respectively. The blocks N_{in} and N_{out} are also included in the DAG, encapsulating the network communication delay model (presented later in Section III-E). More in detail, we can then characterize the sets $\Gamma^L(v) = \{\tau_{S,v}, \tau_{A,v}\}$ and $\Gamma^C(v) = \{\tau_{E,v}, \tau_{F,v}, \tau_{B,v}, \tau_{M,v}\}$ (Figure 3).

We work under the assumption that all tasks follow the read-execute-write paradigm, using global *task labels* for communication and local ones for computation. After a read phase of $\tau_{a,v}$ all its input values are copied from their corresponding global labels to local labels of $\tau_{a,v}$, so that the current execution of $\tau_{a,v}$ works only with the values stored in its local copies. After the write phase of $\tau_{a,v}$, its outputs are written from local labels to their dedicated global labels, so that the data is made available to be read as the input by its successor task in the DAG. Such mechanism can be obtained, e.g., with implicit communication or Logical Execution Time (LET) protocols [46], the exact method left as an open choice.

The DAG encodes functional precedence constraints in *ideal conditions*. If a predecessor task has not completed its current execution or has not executed at all, its successor (on its periodic activation) will operate on the last valid data snapshot stored in the global input task label, thus processing anyway, but with possibly degraded performance since it is acting on stale data. Crucially, we do not explicitly enforce such DAG constraints to be active all the time: It is part of the Challenge to understand under which conditions they are required, to guarantee the given performance requirements.

For the context of this Challenge, for each task (base) periods and execution times are given, while their corresponding deadlines, offsets and priorities (together with the scheduling strategy) are left as design parameters. Numerical values and hardware specifications for the considered platforms are provided in the Challenge code and attached documentation. Additional interfering tasks or synthetic loads may be considered both in-vehicle and in-cloud. We leave this aspect of the Challenge open: In particular, our old Challenges [47], [48] can be used as inspiration to characterize a realistic additional task set configuration in the vehicles.

D. Cloud platform model

The cloud platform executes the control functions of all different vehicles $\Gamma^C = \bigcup_{v \in \mathcal{V}} \Gamma^C(v)$ in parallel. The cloud-based controller infrastructure is allocated a finite and fixed computational capacity, represented by a set of N_c processing cores, dictating an upper bound on the total number of concurrently executing tasks across all vehicle DAGs.

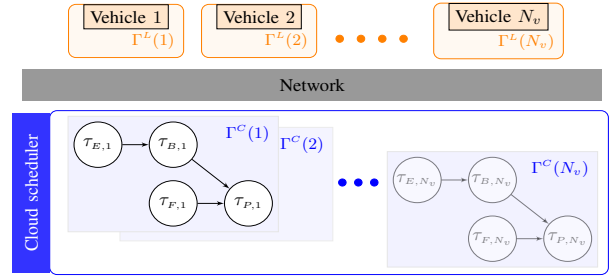


Fig. 3. Mapping view: The cloud scheduler co-schedules and maps the DAGs corresponding to different vehicles

We assume that a dedicated cloud scheduler orchestrates the assignment of computational tasks for the entire set of connected vehicles $\mathcal{V}$, with the goal to optimize their overall control performance. For each vehicle v , its corresponding estimated performance and criticality metrics are computed in-cloud by the estimator task $\tau_{E,v}$ (see again Figure 1). Each estimator task analyzes its vehicle's performance metrics and its operational context in the track, determining whether it is in a critical section (like navigating a sharp curve) or not, as well as if it is experiencing large deviations or behaving as required (e.g., correctly driving within an acceptable lateral error).

Such values can then be provided as inputs to the cloud scheduler, which can use them to take informed decisions at runtime, following a *context-aware resource allocation strategy*, whose detailed design is one of the core targets of this Challenge. Based on a real-time assessment for each vehicle, the cloud scheduler may, e.g., dynamically adjust priorities across the tasks of $\mathcal{V}$, that is, if a vehicle is deemed to be in a critical state, its essential control tasks are immediately given precedence. The scheduler could enforce this priority using two key techniques: *preemptive scheduling*, which interrupts lower-priority tasks (even from other vehicles) to execute urgent ones, and *graceful degradation*, which involves skipping or aborting non-essential tasks for vehicles in safe environments. While this may result in an affected vehicle to temporarily operate on older, “stale” data—a trade-off known as *selective controlled degradation*—it guarantees that the vehicles' most critical control functions always receive the computational resources required to operate without fail.

E. Network communication model

The Challenge includes a vehicle-to-cloud communication architecture, where the vehicles in $\mathcal{V}$ transmit fixed-size sensing messages to a central cloud endpoint, namely N_{in} . Actuation messages are then sent from N_{out} back to the corresponding vehicles. This network is assumed to be unreliable, subject to stochastic delays and packet drops, leading to effects like *message coalescing*, where multiple messages arrive in rapid succession or are aggregated into a single packet [49]. We leave ample freedom to decide the network protocol, under the constraint of choosing realistic bandwidth, such that communication delays cannot be considered as negligible. Under the base assumption of periodic tasks in $\Gamma(v)$, the tasks in

TABLE I
RECAP OF THIS CHALLENGE PARAMETERS

Object	Fixed parameters	Design parameters	Comments
Vehicles set $\mathcal{V}$	Physical model	N_v	-
Taskset Γ	Functional model of $\Gamma(v)$, global and local task labels	-	DAG constraints are not strict for every driving condition
Local vehicle	$\Gamma^L(v)$	Local scheduler, communication protocol	Additional synthetic tasks can be included, e.g., from previous Challenges [47], [48]
$\tau_{S,v}$	$\{T_{S,v}, C_{S,v}\}$	$\{D_{S,v}, \phi_{S,v}, \pi_{S,v}\}$	-
$\tau_{A,v}$	$\{T_{A,v}, C_{A,v}\}$	$\{D_{A,v}, \phi_{A,v}, \pi_{A,v}\}$	-
Cloud platform	Γ^C, N_c	Cloud scheduler, communication protocol, resource allocation, adaptation mechanism	Additional synthetic tasks can be included
$\tau_{E,v}$	$\{T_{E,v}, C_{E,v}\}$	$\{D_{E,v}, \phi_{E,v}, \pi_{E,v}\}$	Computes performance for cloud scheduler
$\tau_{F,v}$	$\{T_{F,v}, C_{F,v}\}$	$\{D_{F,v}, \phi_{F,v}, \pi_{F,v}\}$	-
$\tau_{B,v}$	$\{T_{B,v}, C_{B,v}\}$	$\{D_{B,v}, \phi_{B,v}, \pi_{B,v}\}$	-
$\tau_{M,v}$	$\{T_{M,v}, C_{M,v}\}$	$\{D_{M,v}, \phi_{M,v}, \pi_{M,v}\}$	-
Network communication	N_{in} as single endpoint, message sizes	Protocol, delay model	Additional synthetic messages can be included Realistic bandwidth model should be considered
Controlled CPS	Control tuning, track, simulation time granularity, performance requirements	$v_{0,v}, x_{0,v}$ (from given set)	Adaptive controller in next extensions

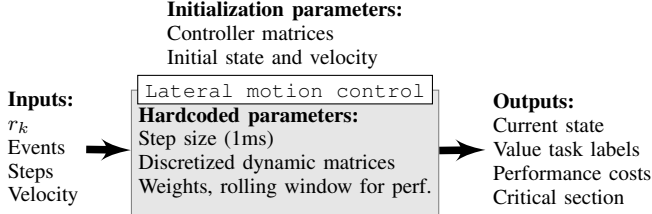


Fig. 4. FMU inputs, outputs and core parameters for a vehicle v

the cloud work on certain snapshots of available sensor inputs sampled periodically. As a consequence, the communication delays can be modeled as a set of discrete values following, e.g., a Markov chain characterization, and are generally more conservative. On the upside, efficient delay-aware controllers can be devised for such models, see for example [40], [39].

On the other hand, by relaxing the hypothesis of periodic tasks in $\Gamma(v)$, more refined communication delay models can be used. Data-driven task chains can reduce reaction time, but introduce challenges with processing jitter and managing inconsistent message arrival rates. In either of these approaches, other key aspects like the sudden arrival of messages must be dealt with user-selected policies, e.g., with selective dropping (e.g., keeping only the latest sample) to ensure data freshness.

F. FMU for physical simulation: basic properties

To ease the work of real-time engineers and improve flexibility, the functional layer is provided in this Challenge as a Functional Mockup Unit (FMU)². The FMU is designed to simulate the lateral motion control system for an arbitrary vehicle v , as introduced in Section III-A, including the physical

model and the functional control chain. It abstracts from the real-time characterization, i.e., it does not explicitly require the knowledge of the task periods, execution times etc. It only requires, as user-driven inputs, the timed events corresponding to the reading and writing instances of each job of the chain $\Gamma(v)$ —which can be extracted by the corresponding schedules of $\Gamma^L(v)$ and $\Gamma^C(v)$ —and to the transmissions and arrivals of messages in the network. The FMU produces as output at runtime the performance metrics values and other useful data for monitoring and debugging. This setup allows for recursive and adaptable handling on the timing part, while the physical part can be treated as a black-box by the non-control-experts. By design, the FMU is compatible with most popular simulation tools, including Simulink and Modelica.

The structure is organized into three main categories of variables: *initialization parameters*, *inputs*, and *outputs*; along with the *hardcoded parameters* of the physical simulation. Figure 4 shows the basic components of the FMU.

1) *Hardcoded parameters for the physical model*: Hard-coded parameters are fixed within the FMU and define fundamental operational characteristics:

- **Simulation step size**: Defines the discrete time interval for simulation, which corresponds also to the minimum granularity for the real-time triggers.
- **Discretized dynamic matrices**: Pre-defined matrices that capture the system's dynamics in a discrete-time format, based on the simulation step.
- **Weights and thresholds for performance**: Parameters used for calculating the output performance metrics.

2) *Init parameters (user/test-dependent)*: These parameters are configurable at the start of the simulation:

- **Controller matrix**: The value of the control parameters, dependent on the given control basic period $T_{B,v}$.

²The FMI standard, from the Modelica Association Project (<https://fmi-standard.org/>), defines a container and an interface to easily exchange dynamic simulation models across hundreds of tools. It makes use of a combination of XML files, binaries and C code, distributed as a ZIP file.

- **Initial state and vehicle velocity:** Sets the starting vehicle values ($v_{0,v}$ and $x_{0,v}$).
- **Physical noise/disturbance:** Probabilistic description of the disturbances (if any) on the sensed states.

In the context of this Challenge, the values are provided together with the model, but can be updated by expert users.

3) *Inputs:* The inputs to the FMU enable recursive simulation and adaptation:

- **Current feedforward reference:** This input drives the vehicle in open loop over the circuit, by producing the desired lateral deviation and its derivatives, to follow the trajectory over ideal conditions.
- **Event triggers at current time:** Boolean inputs that encode the events of reading and writing instances for the current jobs, as well as messages sending and arrival.
- **Number of steps from previous call:** The time interval that has passed from the last call of the FMU: it allows the simulation to advance iteratively, picking up from the last state where at least one trigger occurred.

4) *Outputs:* The FMU generates various outputs that provide insights into the system's state and performance:

- **Current state:** The current state of the physical vehicle.
- **Values in intermediate task labels:** Current value of labels across the different tasks in the control chain.
- **Performance costs:** Multiple performance metrics, both in-vehicle and in-cloud, e.g., cumulative quadratic error, value thresholds, chance constraints etc.
- **Critical section:** Information related to critical operational phases or events within the system, namely when the vehicle is maneuvering in a curve or in a straight line.

IV. CHALLENGE QUESTIONS

In the following, the core open questions spawning from this Challenge are presented. Additionally to the main target of the specific questions, the users can also freely explore the design space of the other free parameters mentioned in the subsections above, and synthesized in Table I.

Q1: Choose a standard (non-context-aware) scheduling policy for the cloud platform and network communication, such that the highest number of vehicles N_v can be accommodated while satisfying the given performance requirements.

As first step, the users should identify the limit of classic hard-deadline models and classic schedulers, where no functional information of the physical system is used at runtime. This result should serve as baseline comparison, against the more advanced solutions of the next questions.

Q2: Design a context-aware and adaptive resource and/or scheduling policy in-cloud that uses the control-related information of the vehicles available at runtime, such that the highest number N_v of vehicle controllers can be executed in parallel on the available N_c cloud cores while satisfying the given performance requirements.

In this second step, we ask the users to identify adaptive strategies that make use of the performance and criticality metrics to adapt their policy online by, e.g., dynamic adjustment of

task priorities or resource allocation, such that a larger number N_v with respect to Q1 can be scheduled, while still satisfying the same control performance requirements.

Q3: Given a fixed number of vehicles N_v , find the minimum amount of computational resources in cloud that satisfy the given performance requirements.

In this question, we target the problem of efficient usage of resources, with the goal of minimizing the overall cost of operation in the cloud, while guaranteeing the required control performance requirements. This aspect is of particular importance to obtain cost-effective applications, and free space for additional parallel tasks.

Q4: Identify new timing abstractions that are able to better capture the relationship between variable sensitivity towards control performance requirements to the corresponding adaptive timing constraints.

Classic timing abstractions focus on worst-case scenarios only, hiding the dependencies to dynamic conditions of CPSs. Here, we ask the users to design more nuanced abstractions, which may, e.g., enforce stricter constraints in specific operating conditions while allowing skipped jobs or missed deadlines in other ones.

Q5: Analyze the resulting end-to-end latency metrics of the control chains, under different scheduling and communication strategies.

Since control software increasingly relies on combining multiple steps in functional chains, understanding how uncertain timing on one or multiple tasks of the chain may affect the whole functionality is of critical importance. From a control perspective, particularly interesting is providing models in terms of *data age* of the applied control commands.

Q6: Relax the assumption of periodic tasks in $\Gamma^C(v)$ and propose instead a data-driven, context-aware and adaptive strategy that solves the previous questions.

As mentioned in Section III-E, latency metrics can be potentially reduced by allowing data-driven triggers for the in-cloud tasks. Freeing this design direction for the previous questions can also potentially improve the overall results.

V. EXAMPLE OF USAGE

This Section presents a small use-case to exemplify the simulations obtainable with the functional model of the FMU presented in the Challenge, and to highlight how timing effects may impact the control performance in non-trivial ways.

A. Simulating a vehicle subject to variable timing delays

For the sake of convenience, we limit this exemplary use-case to a single vehicle v running along the track. Figure 5 shows the overall track in global coordinates (X, Y) , driven in counterclockwise direction, where $(0, 0)$ represents the starting position of v . The track features multiple cornering maneuver with different curvatures as well as a double lane change (from $(0, 20)$ to $(-30, 20)$ in the figure). Such maneuvers are characterized by a desired lateral acceleration $a_{des}(t)$ of the path, and a longitudinal speed $v(t)$, starting from $v_{0,v} = 10$

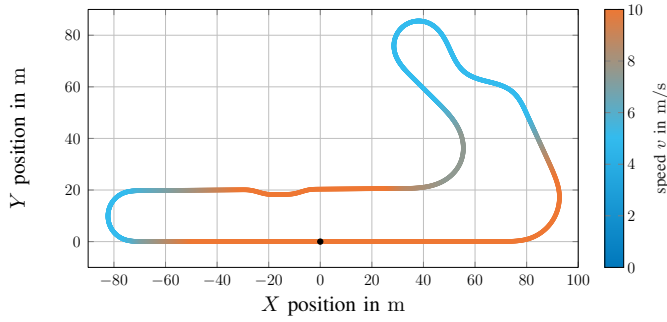


Fig. 5. Considered track with an exemplary speed profile v (colorcoded), the starting point $X = Y = 0$ m, and a counterclockwise driving direction.

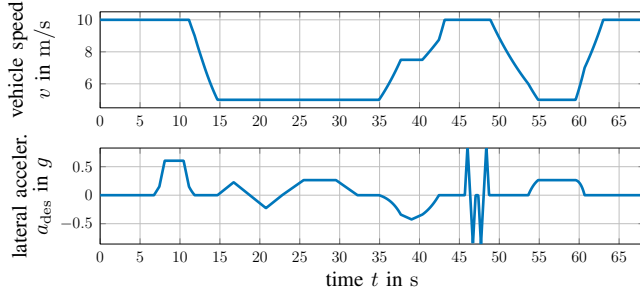


Fig. 6. Exemplary speed profile along the track shown in Fig. 5 (top) and corresponding lateral desired acceleration in multiples of the gravitational constant $g = 9.81 \text{ m/s}^2$ (bottom).

m/s, which is adapted when approaching and leaving a curve, as shown in Fig. 6. Due to the high speed of the exemplary trajectory, the double lane change is the most challenging part of the track with lateral accelerations exceeding $0.5g$.

We consider all tasks in the chain $\Gamma(v)$ as periodic, with periods as in the example of Figure 2 and implicit deadlines. We further assume all tasks being synchronously released, and scheduled with a Rate Monotonic scheduler both for the in-vehicle and in-cloud subsets. We assume that every job reads its input labels at the activation, and writes the output labels at its completion instants, respectively. The simulation performed for this use-case features two distinct setups:

- 1) an ideal condition simulation, where all tasks have execution time of 1ms and are thus always schedulable; and
- 2) an overloaded case, where task $\tau_{B,v}$ can randomly miss deadlines following a weakly-hard constraint of at most 4 consecutive deadline misses in a row, with “kill and hold” strategy [50].

B. Performance requirements

Multiple control performance requirements can be defined in this setup. A requirement consists of a combination between a specific metric, generally a function of the the system state, to be compared against a constraint, i.e., some threshold values. Metrics can be used in the cost function of an optimization problems, and/or as explicit constraints in the formulation, in order to find the best design relative to the chosen formulation.

A typical constraint for lateral vehicle control concerns limiting the lateral error $e(t)$, which is computed as the

difference between the lateral position and the desired path. The lateral error and its first derivative are part of the state vector provided in the FMU formulation (see Appendix A). For example, the cumulative squared lateral error over a sliding window can be used to detect localized violations (e.g., to distinguish between curves and straight parts of the track). Considering a discretized version of the lateral error as e_k , rolling window W and threshold s_W , the formulation becomes:

$$\sum_{i=k-W+1}^k e_i \leq s_W.$$

As another example in a stochastic representation of the scenarios, we can identify if the lateral error exceeds a certain threshold e^* with probability $1 - \beta$. This leads to a so-called *chance constraint* of the following form, which needs to be satisfied at all iterations k :

$$\Pr[|e_k| \leq e^*] \leq 1 - \beta$$

A typical choice is a threshold of $e^* = 20 \text{ cm}$ with a failure probability of $\beta = 0.05$.

C. Simulation results

Exemplary results for the lateral error $e(t)$ of the target vehicle with respect to the desired path are shown in Figure 7 for the two setups (ideal case in blue curve, and 10 realizations of the overloaded setup as orange curves). For instance, we observe that the deadline misses have a minimal impact on the lateral error of only a few centimeters for the straight parts of the track (blue background, bottom left graph in Figure 7). However, they lead to a significant increase of the lateral error during the start and end of the curves. During the double lane change the deviation exceeds the threshold of 20 cm for various realizations of timing variations (green background, bottom right graph in Figure 7).

The results demonstrate that the basic concept of deadline is not enough to characterize the impact of timing variations on the control performance. In particular, also weakly hard constraints prove here to be conservative, since the same number of missed deadlines can lead to very different effects on the performance, depending on both the actual patterns of timing violations and the current state of the system.

VI. CONCLUSIONS

This paper highlights the inadequacy of traditional, static real-time models for modern distributed Cyber-Physical Systems, which often leads to inefficient, over-provisioned designs by ignoring the physical dynamics of the real world. We propose a paradigm shift towards dynamic, context-aware requirements rooted in functional system performance. To drive this change, we introduce the Physics-Driven Real-Time CPS Challenge, providing a realistic automotive simulation for researchers to develop scheduling strategies that leverage physical-state information at runtime.

Exemplary experiments highlight that the impact of timing violations is highly context-dependent, demonstrating that

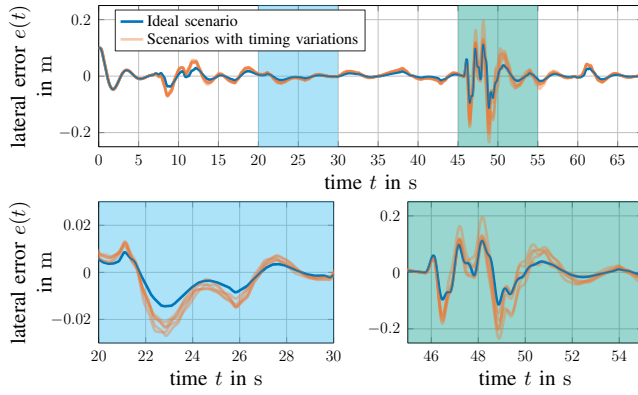


Fig. 7. Trajectory of the lateral error, in case of ideal timing conditions (blue) and with $\tau_{B,v}$ subject to at most 4 deadline misses in a row (orange). For parts of the track where the impact of timing variations is minor (blue background, bottom left) or major (green background, bottom right).

simple deadline-based models established in the real-time community are insufficient to build efficient systems. For instance, the effects of the same number of deadline misses can be irrelevant in one scenario but catastrophic in another. We invite the real-time community to engage with this challenge to develop next-generation resource management techniques and be inspired towards new research directions. The goal is to create more efficient and robust systems by aligning real-time design with the dynamic needs of the physical world.

Furthermore, the challenge is intentionally designed as a living benchmark, intended to be extended in the future. Planned enhancements include incorporating advanced adaptive multi-mode control strategies, richer network models with specific protocols, and more complex environmental and hardware dynamics.

ACKNOWLEDGMENT

The authors would like to thank Dr. Dakshina Dasari for her invaluable inputs on the cloud and network model.

REFERENCES

- [1] G. Bernat, A. Burns, and A. Liamsi, "Weakly hard real-time systems," *IEEE Trans. Comput.*, vol. 50, no. 4, pp. 308–321, 2001.
- [2] B. Akesson, M. Nasri, G. Nelissen, S. Altmeyer, and R. I. Davis, "A comprehensive survey of industry practice in real-time systems," *Real-Time Systems*, vol. 58, no. 3, pp. 358–398, 2022.
- [3] Q. Zhu and A. Sangiovanni-Vincentelli, "Codesign methodologies and tools for cyber-physical systems," *Proceedings of the IEEE*, vol. 106, no. 9, pp. 1484–1500, 2018.
- [4] Q. Zhu, W. Li, H. Kim, Y. Xiang, K. Wardega, Z. Wang, Y. Wang, H. Liang, C. Huang, J. Fan *et al.*, "Know the unknowns: Addressing disturbances and uncertainties in autonomous systems," in *Proc. of the 39th International Conference on Computer-Aided Design*, 2020, pp. 1–9.
- [5] K.-E. Årzén, A. Cervin, J. Eker, and L. Sha, "An introduction to control and scheduling co-design," in *Proc. of the 39th IEEE Conference on Decision and Control*, vol. 5. IEEE, 2000, pp. 4865–4870.
- [6] A. Cervin and J. Eker, "Feedback scheduling of control tasks," in *Proc. of the 39th IEEE Conference on Decision and Control*, vol. 5. IEEE, 2000, pp. 4871–4876.
- [7] A. Cervin, J. Eker, B. Bernhardsson, and K.-E. Årzén, "Feedback-feedforward scheduling of control tasks," *Real-Time Systems*, vol. 23, no. 1, pp. 25–53, 2002.

- [8] D. Fontanelli, L. Greco, and L. Palopoli, "Soft real-time scheduling for embedded control systems," *Automatica*, vol. 49, no. 8, pp. 2330–2338, 2013.
- [9] P. Marti, C. Lin, S. A. Brandt, M. Velasco, and J. M. Fuertes, "Optimal state feedback based resource allocation for resource-constrained control tasks," in *25th IEEE International Real-Time Systems Symposium*. IEEE, 2004, pp. 161–172.
- [10] L. Abeni, T. Cucinotta, G. Lipari, L. Marzario, and L. Palopoli, "Qos management through adaptive reservations," *Real-Time Systems*, vol. 29, no. 2, pp. 131–155, 2005.
- [11] A. Cervin, M. Velasco, P. Marti, and A. Camacho, "Optimal online sampling period assignment: Theory and experiments," *IEEE Transactions on Control Systems Technology*, vol. 19, no. 4, pp. 902–910, 2010.
- [12] A. Cervin, D. Henriksson, B. Lincoln, J. Eker, and K.-E. Årzén, "How does control timing affect performance? analysis and simulation of timing using jitterbug and truetype," *IEEE control systems magazine*, vol. 23, no. 3, pp. 16–30, 2003.
- [13] F. Cremona, M. Morelli, and M. Di Natale, "Tres: a modular representation of schedulers, tasks, and messages to control simulations in simulink," in *Proc. of the 30th Annual ACM Symposium on Applied Computing*, 2015, pp. 1940–1947.
- [14] A. Cervin, P. Pazzaglia, M. Barzegaran, and R. Mahfouzi, "Using jittertime to analyze transient performance in adaptive and reconfigurable control systems," in *24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2019, pp. 1025–1032.
- [15] C. Brandberg and M. Di Natale, "A simevents model for the analysis of scheduling and memory access delays in multicores," in *2018 IEEE 13th International Symposium on Industrial Embedded Systems (SIES)*. IEEE, 2018, pp. 1–10.
- [16] S. Ravi, L. Beermann, O. Kotte, P. Pazzaglia, M. Vinnakota, D. Ziegenbein, and A. Hamann, "Timing-aware software-in-the-loop simulation of automotive applications with fmi 3.0," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2023, pp. 62–72.
- [17] P. Pazzaglia, L. Pannocchi, A. Biondi, and M. Di Natale, "Beyond the weakly hard model: Measuring the performance cost of deadline misses," in *30th Euromicro Conference on Real-Time Systems (ECRTS 2018)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 10–1.
- [18] H. Liang, Z. Wang, R. Jiao, and Q. Zhu, "Leveraging weakly-hard constraints for improving system fault tolerance with functional and timing guarantees," in *Proc. of the 39th International Conference on Computer-Aided Design*, 2020, pp. 1–9.
- [19] C. Hobbs, B. Ghosh, S. Xu, P. S. Duggirala, and S. Chakraborty, "Safety analysis of embedded controllers under implementation platform timing uncertainties," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 4016–4027, 2022.
- [20] P. Skarin, W. Tärneberg, K.-E. Årzén, and M. Kihl, "Towards mission-critical control at the edge and over 5g," in *2018 IEEE International Conference on Edge Computing (EDGE)*. IEEE, 2018, pp. 50–57.
- [21] P. Skarin, W. Tärneberg, K.-E. Årzén, and M. Kihl, "Control-over-the-cloud: A performance study for cloud-native, critical control systems," in *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2020, pp. 57–66.
- [22] P. Mundhenk, A. Hamann, A. Heyl, and D. Ziegenbein, "Reliable distributed systems," in *Proc. 2022 Design, Automation & Test in Europe Conf. & Exhibition (DATE)*. IEEE, 2022, pp. 287–291.
- [23] M. Gallant, K. Schmidt, S. Reimann, F. Berkel, N. Majer, and S. Hohmann, "Distributed model predictive control for cooperative autonomous lane merging," in *2024 European Control Conference (ECC)*. IEEE, 2024, pp. 2704–2711.
- [24] R. Andreoli, R. Mini, P. Skarin, H. Gustafsson, J. Harmatos, L. Abeni, and T. Cucinotta, "A multi-domain survey on time-criticality in cloud computing," *IEEE Transactions on Services Computing*, 2025.
- [25] M. S. Mahmoud and M. M. Hamdan, "Fundamental issues in networked control systems," *IEEE/CAA Journal of Automatica Sinica*, vol. 5, no. 5, pp. 902–922, 2018.
- [26] P. Pazzaglia and M. Maggio, "Characterizing the effect of deadline misses on time-triggered task chains," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 3957–3968, 2022.
- [27] M. Guenzel, H. Teper, G. v. d. Brüggem, and J.-j. Chen, "End-to-end latency of cause-effect chains: a tutorial," *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 1, pp. 1–18, 2024.

- [28] M. Jelali, "An overview of control performance assessment technology and industrial applications," *Control Engineering Practice*, vol. 14, no. 5, pp. 441–466, 2006.
- [29] P. Pazzaglia, C. Mandrioli, M. Maggio, and A. Cervin, "DMAC: Deadline-miss-aware control," in *31st Euromicro Conference on Real-Time Systems (ECRTS 2019)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2019, p. 1.
- [30] K. J. Åström and B. Wittenmark, *Computer-controlled systems: theory and design*. Courier Corporation, 2013.
- [31] K. G. Shin and X. Cui, "Computing time delay and its effects on real-time control systems," *IEEE Transactions on control systems technology*, vol. 3, no. 2, pp. 218–224, 1995.
- [32] D. Hrovat, S. Di Cairano, H. Tseng, and I. Kolmanovsky, "The development of model predictive control in automotive industry: A survey," in *2012 IEEE International Conference on Control Applications*, 2012, pp. 295–302.
- [33] M. B. Saltık, L. Özkan, J. H. Ludlage, S. Weiland, and P. M. Van den Hof, "An outlook on robust model predictive control algorithms: Reflections on performance and computational aspects," *Journal of Process Control*, vol. 61, pp. 77–102, 2018.
- [34] C. Lu, J. A. Stankovic, S. H. Son, and G. Tao, "Feedback control real-time scheduling: Framework, modeling, and algorithms," *Real-Time Systems*, vol. 23, no. 1, pp. 85–126, 2002.
- [35] L. Hewing, J. Kabzan, and M. N. Zeilinger, "Learning-based model predictive control: Toward safe learning in control," *Annual Review of Control, Robotics, and Auton. Systems*, vol. 3, no. 1, pp. 269–296, 2020.
- [36] A. Lahr, A. Zanelli, A. Nurkanović, and M. Diehl, "L4acados: Learning-based models for acados, applied to gaussian process-based predictive control," *arXiv preprint arXiv:2411.19258*, 2024.
- [37] P. Marti, J. M. Fuertes, G. Fohler, and K. Ramamritham, "Jitter compensation for real-time control systems," in *Proc. 22nd IEEE Real-Time Systems Symposium (RTSS 2001)*. IEEE, 2001, pp. 39–48.
- [38] A. Cervin, B. Lincoln, J. Eker, K.-E. Årzén, and G. Buttazzo, "The jitter margin and its application in the design of real-time control systems," in *Proc. of the 10th international conference on real-time and embedded computing systems and applications*, 2004.
- [39] M. Gallant, C. Mark, P. Pazzaglia, J. von Keler, L. Beermann, K. Schmidt, and M. Maggio, "Structure-exploiting distributionally robust control of non-homogeneous markov jump linear systems," *IEEE Control Systems Letters*, 2024.
- [40] C. Mark, P. Pazzaglia, and K. Schmidt, "Distributionally robust output-feedback control of markov jump linear systems," in *2024 American Control Conference (ACC)*. IEEE, 2024, pp. 4705–4710.
- [41] P. Pazzaglia, A. Hamann, D. Ziegenbein, and M. Maggio, "Adaptive design of real-time control systems subject to sporadic overruns," in *Proc. 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 1887–1892.
- [42] N. Vreman, C. Mandrioli, and A. Cervin, "Deadline-miss-adaptive controller implementation for real-time control systems," in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp. (RTAS)*, 2022, pp. 13–26.
- [43] D. Henriksson and A. Cervin, "Optimal On-Line Sampling Period Assignment for Real-Time Control Tasks Based on Plant State Information," in *IEEE Conf. Decis. Control (CDC)*, 2005, pp. 4469–4474.
- [44] M. Domenighini, P. Pazzaglia, C. Mark, K. Schmidt, L. Beermann, and A. V. Papadopoulos, "Control period adaptation for resource-constrained mpc applications," in *2025 European Control Conference (ECC)*. IEEE, 2025, pp. 2255–2260.
- [45] M. Gallant, C. Mark, P. Pazzaglia, J. von Keler, L. Beermann, K. Schmidt, and M. Maggio, "Soft-constrained stochastic mpc of markov jump linear systems: Application to real-time control with deadline overruns," *IEEE Control Systems Letters*, 2025.
- [46] A. Hamann, D. Dasari, S. Kramer, M. Pressler, and F. Wurst, "Communication centric design in complex automotive embedded systems," in *29th Euromicro Conference on Real-Time Systems (ECRTS 2017)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2017, pp. 10–1.
- [47] A. Hamann, D. Dasari, S. Kramer, M. Pressler, F. Wurst, and D. Ziegenbein, "Waters industrial challenge 2017," in *International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems (WATERS)*, 2017.
- [48] A. Hamann, D. Dasari, S. Kramer, M. Pressler, F. Wurst, D. Ziegenbein, I. Sañudo, N. Capodiceci, P. Burgio, and M. Bertogna, "Waters industrial challenge 2019," in *International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems (WATERS)*, 2019.
- [49] D. Shan, F. Ren, P. Cheng, R. Shu, and C. Guo, "Observing and mitigating micro-burst traffic in data center networks," *IEEE/ACM Transactions on Networking*, vol. 28, no. 1, pp. 98–111, 2020.
- [50] A. Cervin, "Analysis of overrun strategies in periodic control tasks," *IFAC Proceedings Volumes*, vol. 38, no. 1, pp. 219–224, 2005.
- [51] W. Zhai, K. Wang, and C. Cai, "Fundamentals of vehicle-track coupled dynamics," *Vehicle System Dynamics*, vol. 47, no. 11, pp. 1349–1376, 2009.
- [52] M. Corno, G. Panzani, and S. M. Savaresi, "Single-track vehicle dynamics control: State of the art and perspective," *IEEE/ASME transactions on mechatronics*, vol. 20, no. 4, pp. 1521–1532, 2015.
- [53] A. Hamann, D. Ziegenbein, S. Kramer, and M. Lukasiewicz, "FMTV 2016 verification challenge," *Inf Process Lett*, 2016.
- [54] S. Kramer, D. Ziegenbein, and A. Hamann, "Real world automotive benchmarks for free," in *International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems (WATERS)*, 2015.
- [55] "Eclipse APP4MC," <https://eclipse.dev/app4mc/>, accessed: 2026-01-13.
- [56] V. Bandur, G. Selim, V. Pantelic, and M. Lawford, "Making the case for centralized automotive e/e architectures," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 2, pp. 1230–1245, 2021.
- [57] S. Saidi, S. Steinhorst, A. Hamann, D. Ziegenbein, and M. Wolf, "Future automotive systems design: Research challenges and opportunities," in *2018 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, 2018, pp. 1–7.
- [58] L. Thames and D. Schaefer, "Software-defined cloud manufacturing for industry 4.0," *Procedia cirp*, vol. 52, pp. 12–17, 2016.
- [59] F. Rehm, D. Dasari, A. Hamann, M. Pressler, D. Ziegenbein, J. Seitter, I. Sañudo, N. Capodiceci, P. Burgio, and M. Bertogna, "Performance modeling of heterogeneous hw platforms," *Microprocessors and Microsystems*, vol. 87, p. 104336, 2021.

APPENDIX A TECHNICAL MODEL

The physical model consists of a single-track vehicle model, plus a steering and lateral error model (see Figure 8). The single-track vehicle model describes the lateral dynamics of the vehicle in terms of yaw rate $\dot{\psi}$ and side-slip angle β [51]. Based on linear tire characteristics, the model captures the relationship between steering input δ and the resulting lateral motion of the vehicle. The main ordinary differential equations (with implicit dependencies on time t) are

$$\ddot{\psi} = \frac{-c_f l_f^2 - c_r l_r^2}{J_z} \dot{\psi} + \frac{c_r l_r - c_f l_f}{J_z} \beta + \frac{c_f l_f}{J_z} \delta \quad (1a)$$

$$\dot{\beta} = \left(\frac{-c_f l_f + c_r l_r}{mv^2} - 1 \right) \dot{\psi} - \frac{c_f + c_r}{mv} \beta + \frac{c_f}{mv} \delta, \quad (1b)$$

where c_f and c_r are the cornering stiffness of the front and rear tires, respectively, l_f and l_r are the distances from the vehicle's center of gravity to the front and rear axles, respectively, J_z is the yaw moment of inertia, m is the vehicle mass, and v is the current longitudinal velocity.

The steering model is a second-order system which models how the actual steering angle δ reacts to a desired steering angle δ_{des} . It is defined as:

$$\ddot{\delta} = \omega_s^2 (\delta_{\text{des}} - \delta) - 2d_s \omega_s \dot{\delta}. \quad (1c)$$

The lateral error model describes how e evolves over time based on the desired and actual lateral acceleration:

$$\ddot{e} = a_{\text{des}} - a_{\text{veh}} = (\kappa_{\text{des}} + \kappa'_{\text{des}} L) v^2 - (\dot{\psi} + \dot{\beta}) v. \quad (1d)$$

As shown in Figure 8 the lateral error is defined as the distance between the vehicle's current position with a look ahead distance L and the desired path. The latter is characterized

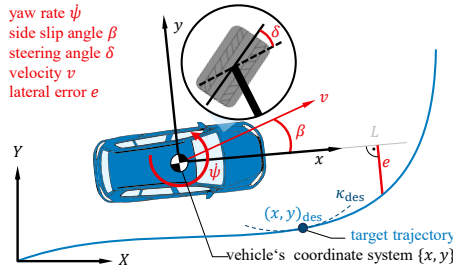


Fig. 8. Illustration of the physical states of the technical model (red) and the target trajectory with the desired curvature κ_{des} (blue).

by its curvature of the desired trajectory κ_{des} and its derivative w.r.t. the longitudinal position $\kappa'_{\text{des}} = d\kappa_{\text{des}}/dx$. The complete physical model is obtained by combining equations (1a)–(1d) with the state $x^T = [\psi, \dot{\psi}, \beta, \delta, \dot{\delta}, e, \dot{e}]$ and the measured output $y^T(t) = [\dot{\psi}, e, \dot{e}, \ddot{e}]$.

The controller $u(t) = u^r(t) + u^c(t)$ features a feedforward and a feedback component given the reference $r^T(t) = [\kappa_{\text{des}}(t), \kappa'_{\text{des}}(t)]$. The feedforward component $u^r(t)$ is designed to follow the desired path based on the reference curvature and its derivative [52]. The feedback component $u^c(t)$ is a feedback controller that stabilizes the closed-loop system and ensures robustness against disturbances and model uncertainties.

$$u = K_1(v)\dot{\psi}_d + K_2(\dot{\psi}_d - \dot{\psi}) \quad (2a)$$

$$\dot{\psi}_d = \dot{\psi}_{d,\text{ff}}(r, v) + v^{-1}(k_1 e + k_2 \dot{e} + k_2 \ddot{e}) \quad (2b)$$

By separating the parts with dependence on the reference and the parts depending on the measured output, we obtain the feedforward and feedback components. The complete closed-loop system can be represented in state-space form as follows:

$$\dot{x} = A(v)x + B(v)u^c + F(v)r \quad (3a)$$

$$y = Cx + \nu \quad (3b)$$

$$u^c = K(v)y \quad (3c)$$

Here ν represents measurement noise. The matrices $A(v)$, $B(v)$, $F(v)$, C , and $K(v)$ are derived from the physical model and controller design, with dependence on the vehicle speed v .

APPENDIX B

REFLECTIONS ON PREVIOUS BOSCH CHALLENGES

Recognizing the risk of an increased gap between industry and research, Bosch has engaged in open and sustained interaction with academia by publishing performance models that capture key characteristics of real industrial applications. These models have served as the basis for the *Industrial Challenges* presented by Bosch at the former WATERS workshop co-located with the ECRTS conference.

A. 2016/2017: Complex μC -based multi-core systems

Initially, performance models targeted “classical real-time systems” featuring μC -based multi-core hardware platforms. Accordingly, the first two Bosch Industrial Challenges

(2016 [53], 2017 [47]) were based on a realistic full-blown modern engine management system [54] and described using the open-source Eclipse project APP4MC [55].

The objective of these initial industrial challenges was to illustrate the complexity of industrial real-time systems. The proposed model combined approximately 20 period- and interrupt-driven tasks, with functionalities spread across over 1250 runnables, called sequentially within these tasks, and featuring more than 10.000 global variables.

System complexity is underscored not only by these metrics but also by the diverse range of real-time constraints involved. These constraints span from classical period-equals-deadline requirements to dynamic deadlines of interrupt-driven angle-synchronous tasks, and cause-effect chains involving multiple runnables mapped to different tasks with varied activation patterns (from interrupt-driven to periodic activation and vice-versa). Notably, end-to-end latency analysis along cause-effect chains in a multi-core setting with intricate data sharing and concurrent execution proved particularly challenging [46]. The response to these industrial challenges was exceptional. The challenge model has been (and continues to be) extensively utilized in numerous papers (several hundreds of citations) for evaluating formal, simulation, and hybrid approaches on a real-world example. Admittedly, most researchers used only the application structure (tasks, runnables, communication, constraints) of the challenge and ignored the fact that the observed WCETs actually overloaded the system, for which we expected instead a probabilistic validation, as well as degradation and failure mitigation measures.

These initial challenges proved highly successful, largely owing to the good alignment of their models with the abstractions commonly used in real-time research. This facilitated straightforward justification for the relevance of various research efforts. However, the computational model presented by these challenges corresponds to mature systems which are nowadays mastered by industry practitioners. Furthermore, these challenges completely ignore functional and physical aspects, which play a crucial role in CPSs. Thus, from Bosch perspective, these two challenges are no longer relevant.

B. 2019: Towards heterogeneous μP -based platforms

The 2019 Industrial Challenge addressed new trends in the design and implementation of automotive E/E (Electrical/Electronic) architectures. Especially, the computational power and communication bandwidth required for novel advanced functionalities, such as automated driving or connected vehicle functions (e.g., perception, maneuver planning, adaptive cruise control), exceed the capabilities of μC -based platforms, focus of the first two industrial challenges. For this reason, automotive systems are currently reorganized following paradigms of *centralized and zone-oriented* E/E architectures [56], based on a new class of computing nodes, featuring more powerful μP cores and specialized accelerators [57]. A similar trend is observable in industrial automation, shifting from μC - to μP -based platforms utilizing POSIX-based operating systems and container technology. This transition

aims to achieve greater flexibility in software deployment within factory environments aligning with the concept of *Software-Defined Manufacturing* [58], enabling more agile and reconfigurable production systems.

In order to address the changing system architectures based on a concrete example, Bosch proposed in cooperation with Marko Bertogna's group at Modena an Industrial Challenge describing the deployment of an autonomous driving application on a Nvidia Tegra X2 platform [48]. The purpose was to explore how state-of-the-art real-time modeling and analysis techniques fitted an application executed on a heterogeneous hardware platform where time critical cause-effect chains span across general purpose μ P cores and GPUs. It was expected that an overall analysis incorporating all aspects (CPU and GPU scheduling, task mapping, interference on the memory hierarchy, synchronous and asynchronous offloading, response time analysis, end-to-end latency analysis) would be challenging using the available toolbox of real-time research.

The results (summarized in [59]), however, showed that this is only partially true. While each of the proposed solutions made some simplifying assumptions for parts of the overall problem, they nevertheless proposed accurate methods for other parts. In summary, the toolbox of real-time research in general fits the proposed challenge. However, still much work remains to be done to develop a holistic approach that covers all the above-mentioned aspects in more detail.

Overall, this Challenge has seen less community uptake compared to the initial two. Nevertheless, its continued relevance remains high, particularly as the heterogeneity of compute platforms is set to increase further, significantly driven, for instance, by the emerging trend towards chiplet systems as well as systems with a compute continuum, spanning from μ C-based devices to cloud [22]. Bosch therefore encourages the real-time community to continue focusing on such increasingly heterogeneous compute platforms.